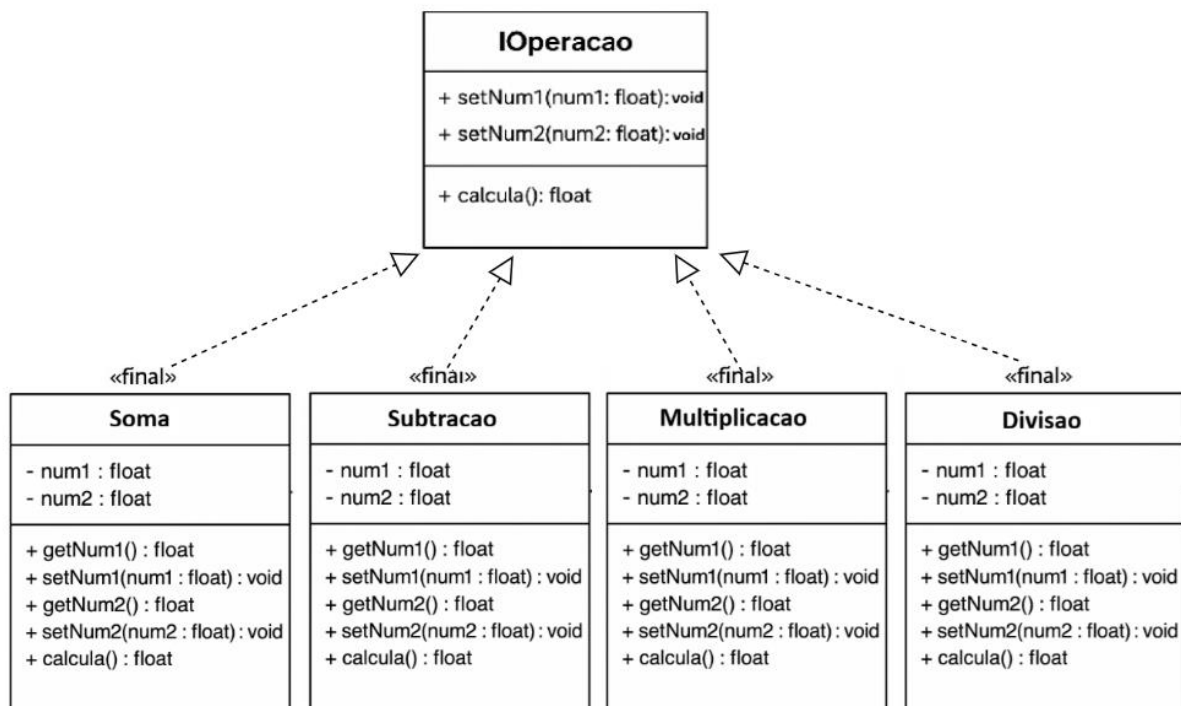


ATENÇÃO! As atividades práticas serão por meio de atividades já solucionadas. Nossa ideia é que você entenda o contexto e recrie o código-fonte apresentado. Com isso, você terá exemplos mais robustos para poder seguir sua evolução.

Desenvolvimento da versão 6 (Herança e Polimorfismo)

Na **versão anterior** de nossa calculadora, **implementamos** o conceito de **interface**, para **obrigar** que as **classes de operação** seguissem um **padrão** em relação às **nomenclaturas de seus métodos**. Dessa forma, se olharmos o diagrama de classes da versão anterior(veja abaixo), sabemos que as classes **Soma**, **Subtracao**, **Multiplicacao** e **Divisao** **contém** um método **calcula()**, e que é usado esse **mesmo nome** para **todas as classes**, de forma que se **usarmos a classe Soma**, **não é necessário** consultar as demais classes para saber o **nome do método correspondente**. Obviamente, em nossa pequena proposta, é muito simples e rápido consultarmos as classes de operação.



Você **sempre** deve pensar nos **conceitos de POO** em larga escala, em situações onde você **possuirá muitas classes**, uma **hierarquia mais complexa**, onde um **diagrama de classes** lhe **auxiliará** e muito a **entender a estrutura**. Nesse contexto,

a **utilização de interfaces**, **se aplicáveis**, permitem a **organização/abstração** que é necessária quando se **programa** nesse paradigma.

Como mencionamos na Atividade anterior, ainda temos bastante repetição de código que podemos otimizar, portanto, nessa versão, abordaremos os conceitos de **Herança** e **Polimorfismo** mais 2 **recursos** da **Programação Orientada a Objetos** **bastante úteis** e **amplamente utilizados**, neste contexto. Entender **como e quando utilizar** Herança e Polimorfismo, além de **conhecer o contexto de sua utilização**, quando você estiver **pesquisando** na documentação oficial do PHP é fundamental.

Aproveitando esta abertura em que soube sobre o novo recurso que iremos utilizar em nossa calculadora, FAÇA UMA PESQUISA E FICHAMENTO SOBRE HERANÇA E POLIMORFISMO ABORDANDO O SEGUINTE:

HERANÇA:

1. Definição de Herança

- O que significa Herança em programação?
- Qual a ideia central por trás do conceito?

2. Finalidade/Importância

- Para que serve usar herança?
- Quais vantagens ela traz?

POLIMORFISMO:

1. Conceito básico:

- O que significa “polimorfismo”?
- Qual a ideia central do polimorfismo na POO?

2. Relação com Herança:

- Como o polimorfismo se relaciona com a herança?
- Por que só faz sentido falar em polimorfismo quando há Herança(ou interface)?

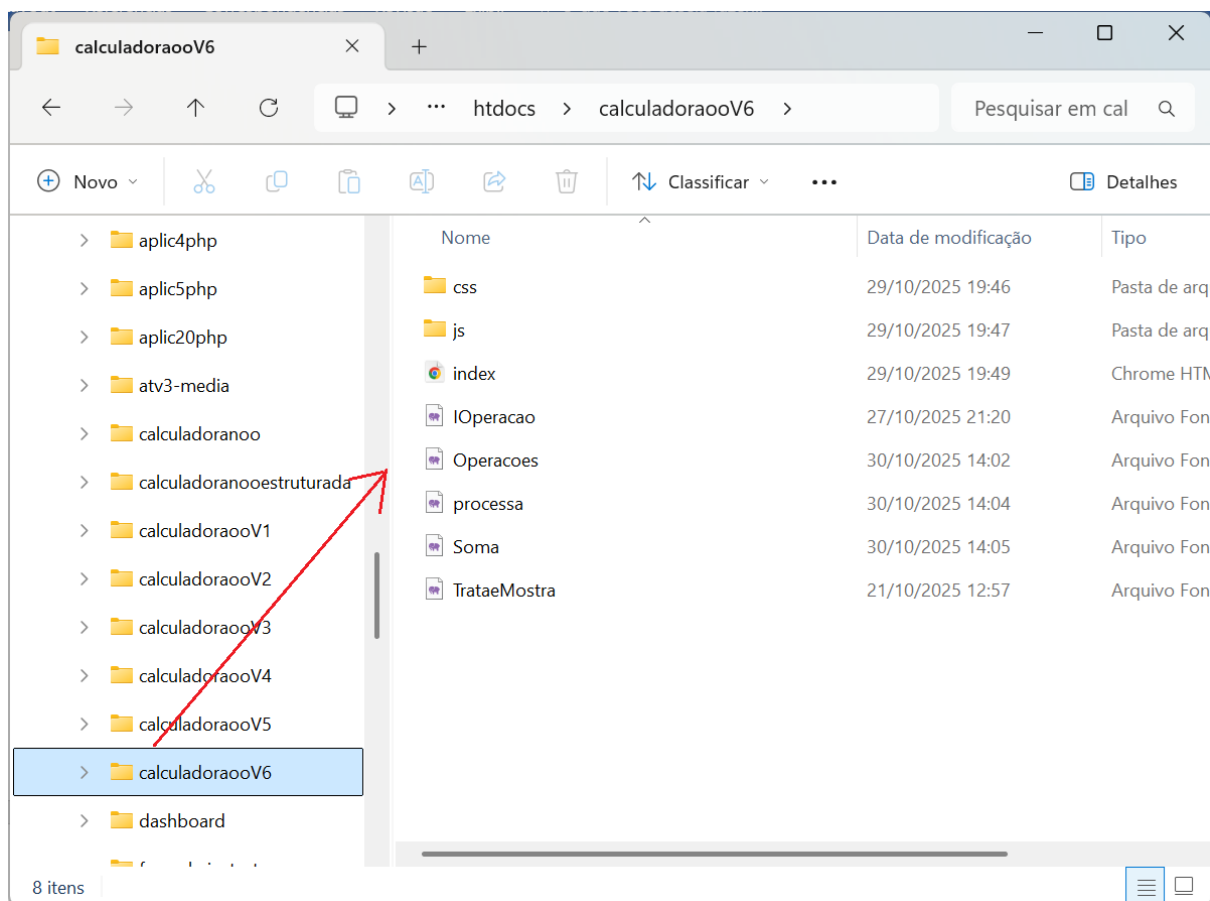
NOTA: FAVOR ALOCAR ESTE FICHAMENTO JUNTAMENTE COM OS ANTERIORES, NAQUELE DOCUMENTO INICIAL EM QUE FEZ OS PRIMEIROS FICHAMENTOS OK?

ATENÇÃO!!!!

- A exemplo das versões anteriores, o **FrontEnd** desta **versão** será o mesmo utilizado **anteriormente**, podendo você fazer o **reaproveitamento dos códigos** dos **scripts** sem **nenhum problema**.
- **A EXEMPLO DA VERSÃO ANTERIOR**, para efeito de melhor **interação** e **fixação** do conteúdo, esta **OT** vai te **orientar** a fazer o **ajuste** apenas para a classe **Soma**, ficando os **demais ajustes** para as classes **Subtracao**, **Multiplicacao** e **Divisao** sobre sua **responsabilidade OK?**
-

Feitas as devidas explicações, vamos iniciar a construção da **versão 6...**

Estrutura de pastas



Fluxo do programa

1. Usuário preenche **valor1**, **valor2** e escolhe a **operação**.
2. **JavaScript** valida os campos com **regex** no cliente. Ele **impede** o envio se **inválido**.
3. Ao **submeter**, o formulário faz **POST** para a mesma página (**index.php**).
4. **PHP** define as classes, **Soma**, **Subtracao**, **Multiplicacao** e **Divisao** com seus respectivos métodos **dinâmicos**(Nesta versão, estas classes estarão relacionadas com uma superclasse(classe mãe) **Operacoes** que, por sua vez, estará relacionada com a interface **IOperacao**) e a classe **TrataeMostra** com os métodos **estáticos**: **exibir os resultados** e **limpar/validar os dados de entrada**.
5. O servidor **converte** entradas, **chama** o método **estático** correspondente e usa **TrataeMostra::exibirResultado()** (método estático) para **montar** a saída HTML.
6. O resultado **aparece na página** quando a resposta for **renderizada**.

INTERFACE

Calculadora - Programação Estruturada

Valor 1

Valor 2

Operação

Somar (+) ▾

CalcularLimpar

Enviar os valores e a operação para `processa.php` que processará a função correspondente.

Note que continua sendo a mesma que utilizamos nas versões anteriores, não temos o porquê mudar, apenas vamos trabalhar, a cada nova versão, com os novos recursos que serão necessários para o aprendizado dos fundamentos da POO.

SCRIPT HTML

```
processa.php  index.html X
index.html > ...
1  <!doctype html>
2  <html lang="pt-BR">
3  <head>
4  <meta charset="utf-8">
5  <meta name="viewport" content="width=device-width,initial-scale=1">
6  <title>Calculadora Estruturada</title>
7  <link rel="stylesheet" type="text/css" href="css/estilo.css">
8  <script src="js/validacoes.js"></script>
9  </head>
10 <body>
11   <main class="container">
12     <h1>Calculadora OO V6- Programação Orientada a Objetos</h1>
13
14     <form id="calcForm" action="processa.php" method="post" onsubmit="return validaFormulario()">
15       <div class="field">
16         <label for="valor1">Valor 1</label>
17         <input type="text" id="valor1" name="valor1" required autocomplete="off">
18       </div>
19
20       <div class="field">
21         <label for="valor2">Valor 2</label>
22         <input type="text" id="valor2" name="valor2" required autocomplete="off">
23       </div>
24
25       <div class="field">
26         <label for="operacao">Operação</label>
27         <select name="operacao" id="operacao">
28           <option value="somar">Somar (+)</option>
29           <option value="subtrair">Subtrair ( )</option>
30           <option value="multiplicar">Multiplicar (x)</option>
31           <option value="dividir">Dividir (÷)</option>
32         </select>
33       </div>
34
35       <div class="actions">
36         <button type="submit">Calcular</button>
37         <button type="reset">Limpar</button>
38       </div>
39     </form>
40
41     <footer>
42       <p>Enviar os valores e a operação para 'processa.php' que processará a função correspondente.</p>
43     </footer>
44   </main>
45 </body>
46 </html>
```

O **script** é o mesmo utilizado no software das versões anteriores da Calculadora. Sem nenhuma novidade, apenas troque o texto da tag **<h1>** para “Calculadora OO V6 - Programação Orientada a Objetos”.

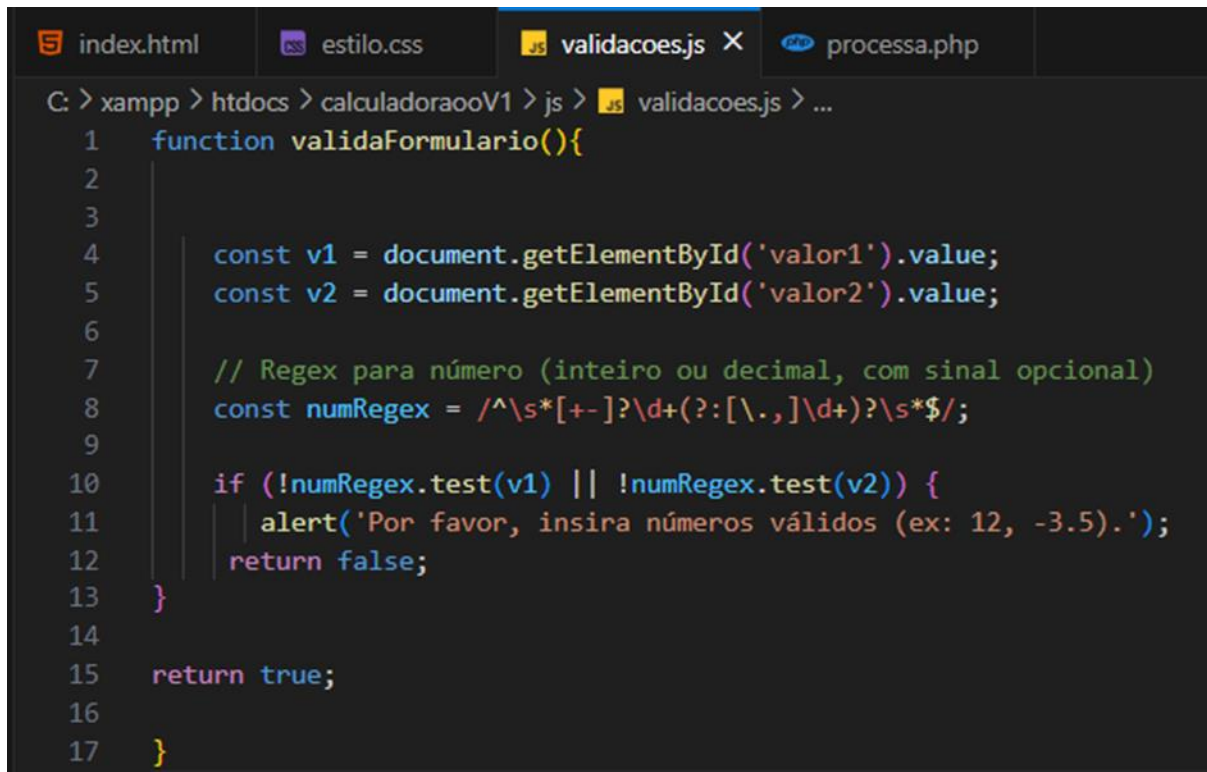
SCRIPT CSS

```
index.html  estilo.css X  validacoes.js  processa.php  Calculadora.php  TrataMostra.php
C:\> xampp > htdocs > calculadoraooV3 > css > estilo.css > root

1  :root{
2    --bg: #ef6f7fb;
3    --card: #fff;
4    --accent: #2b6cb0;
5    --muted: #666
6  }
7  *{
8    box-sizing: border-box;
9    font-family: Inter, system-ui, Arial, sans-serif
10 }
11 body{
12   margin: 0;
13   background: var(--bg);
14   color: #222
15 }
16 .container{
17   max-width: 520px;
18   margin: 48px auto;
19   padding: 24px;
20   background: var(--card);
21   border-radius: 12px;
22   box-shadow: 0 6px 20px #rgba(20,20,40,0.06)
23 }
24 h1{margin: 0 0 18px;
25   font-size: 22px
26 }
27 .field{
28   margin-bottom: 12px
29 }
30 label{
31   display: block;
32   margin-bottom: 6px;
33   font-size: 14px;
34   color: var(--muted)
35 }
36 input[type=“text”], select{
37   width: 100%;
38   padding: 10px;
39   border-radius: 8px;
40   border: 1px solid #ddd;
41   font-size: 16px
42 }
43 .actions{
44   display: flex;
45   gap: 8px
46 }
47 button{
48   padding: 10px 14px;
49   border-radius: 8px;
50   border: 0;
51   background: var(--accent);
52   color: #fff;
53   cursor: pointer
54 }
55 button[type=“reset”]{
56   background: #999
57 }
58 .error{
59   color: #c53030;
60   margin-top: 8px
61 }
62 footer{
63   margin-top: 16px;
64   font-size: 13px;
65   color: var(--muted)
66 }
67
```

O **script** é o **mesmo** utilizado no software da calculadora das **versões anteriores**. Sem nenhuma novidade.

SCRIPT JavaScript



```
index.html  estilo.css  validacoes.js X  processa.php

C: > xampp > htdocs > calculadoraooV1 > js > validacoes.js > ...
1  function validaFormulario(){
2
3
4      const v1 = document.getElementById('valor1').value;
5      const v2 = document.getElementById('valor2').value;
6
7      // Regex para número (inteiro ou decimal, com sinal opcional)
8      const numRegex = /^\\s*[+-]?\\d+(?:[\\.,]\\d+)?\\s*$/;
9
10     if (!numRegex.test(v1) || !numRegex.test(v2)) {
11         alert('Por favor, insira números válidos (ex: 12, -3.5).');
12         return false;
13     }
14
15     return true;
16
17 }
```

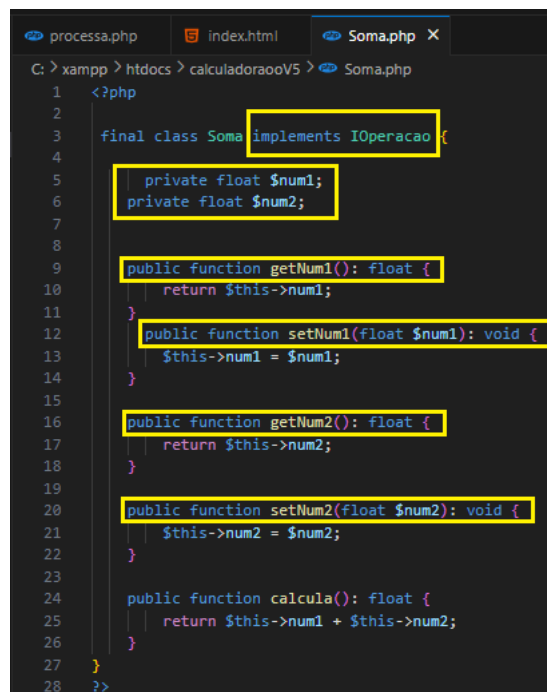
O **script** é o **mesmo** utilizado no software da calculadora das **versões anteriores**. Sem nenhuma novidade.

SCRIPT PHP

Este **Script** requer que **façamos algumas explicações** pois trás os **conceitos bem importantes da POO**, como já relatamos e também você já fez pesquisa, trata-se da **Herança e Polimorfismo**. Sendo assim, **faremos** a escrita deste Script de maneira um pouco **diferenciada** das anteriores, estaremos fazendo esta implementação de **forma mais detalhada e narrativa** para que possa **compreender** bem os **conceitos** e os **por quês** do que estaremos fazendo, então, vamos em frente.

Como mencionamos na **Atividade** anterior, ainda temos **bastante repetição de código** que podemos **otimizar**. Em seu fichamento sobre **Herança** você viu que uma **classe mãe** tem o papel de **fornecer às suas classes filhas recursos (métodos e atributos) comuns a elas**, evitando assim a **repetição de código**. O que faremos nessa versão é **analisarmos** tudo o que é **igual** e **está se repetindo** nas classes de **operação** e implementarmos isso uma **única vez** na classe **Operacoes**. Simmm! **Ela voltou!** No print a seguir, está **sinalizado** tudo o que se **repete** nas **quatro classes de operações** que temos: **Soma**, **Subtracao**, **Multiplicacao** e **Divisao**, tomando como exemplo, só para variar a classe **Soma**:

O SCRIPT ABAIXO AINDA NÃO É O OFICIAL PARA ESTA SOLUÇÃO, SERVINDO PARA ILUSTRAÇÃO DO QUE SERÁ REMANEJADO PARA A CLASSE MÃE!!



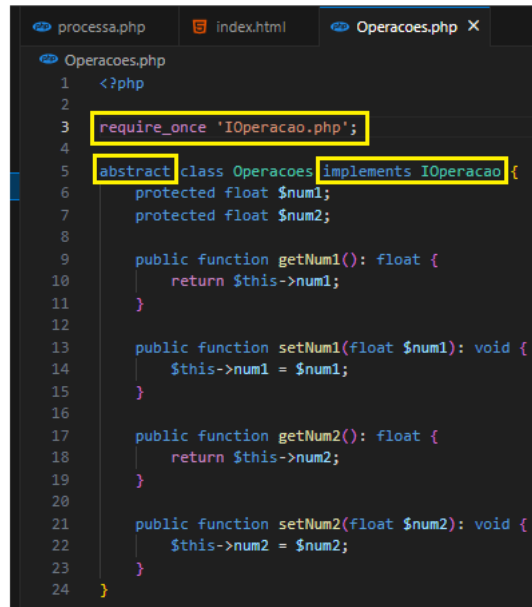
```
1 <?php
2
3 final class Soma implements IOperacao {
4
5     private float $num1;
6     private float $num2;
7
8
9     public function getNum1(): float {
10         return $this->num1;
11     }
12     public function setNum1(float $num1): void {
13         $this->num1 = $num1;
14     }
15
16     public function getNum2(): float {
17         return $this->num2;
18     }
19
20     public function setNum2(float $num2): void {
21         $this->num2 = $num2;
22     }
23
24     public function calcula(): float {
25         return $this->num1 + $this->num2;
26     }
27 }
28 ?>
```

Mais precisamente, tudo o que temos **nas classes de operação**, marcado em **amarelo**, começando pela **implementação da interface IOperacao** é **repetido** em **todas as classes**. O único ponto **diferente** é o **método calcula()**. Apesar de que **por conta da interface** ele contenha o **mesmo nome** em **todas as classes**, sua **implementação** é **diferente** em **cada uma delas** e por **conta disso não podemos** considerá-lo como **algo repetido**. A **ideia** então é **extrair das classes de**

operação tudo o que identificamos e **inserir na classe mãe Operacoes**. Vamos lá?

HERANÇA

SCRIPT CLASSE MÃE – Operacoes.php



```
1 <?php
2
3 require_once 'IOperacao.php';
4
5 abstract class Operacoes implements IOperacao {
6     protected float $num1;
7     protected float $num2;
8
9     public function getNum1(): float {
10         return $this->num1;
11     }
12
13     public function setNum1(float $num1): void {
14         $this->num1 = $num1;
15     }
16
17     public function getNum2(): float {
18         return $this->num2;
19     }
20
21     public function setNum2(float $num2): void {
22         $this->num2 = $num2;
23     }
24 }
```

Deixamos em destaque algumas coisas para que você preste mais atenção.

DETALHANDO!!!!

`require_once 'IOperacao.php';` - Como já explicado anteriormente, serve para **importar** o conteúdo, neste caso, do arquivo '**IOperacao**' que em nossa solução é a nossa **interface**, considerando que agora a **classe mãe**, **Operacoes**, é que estará **vinculada a interface**. Importando a interface este vínculo pode ocorrer, do contrário, não teria condições.

ATENÇÃO: APESAR DE NÃO TERMOS SOLICITADO, ACREDITAMOS QUE FIQUE CLARO PARA VOCÊ QUE

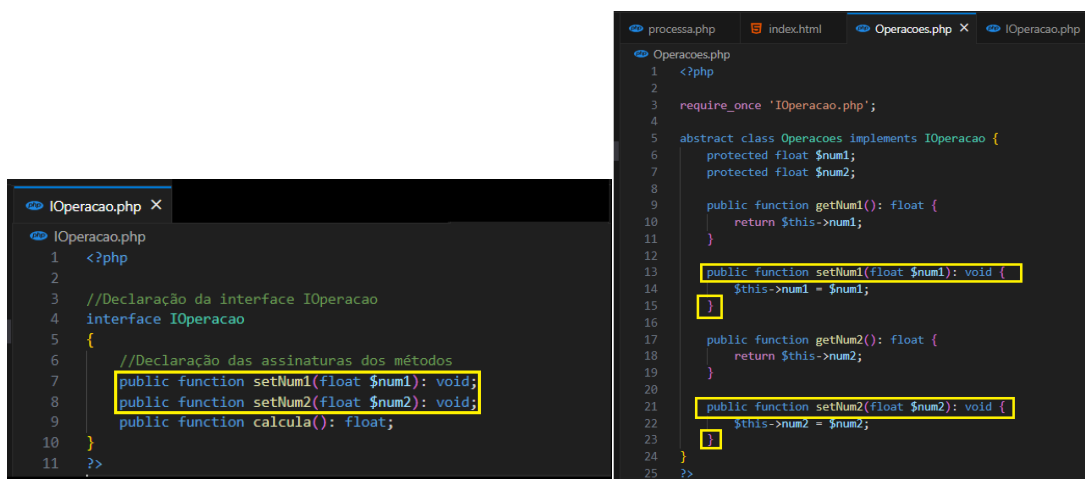
TERÁ TAMBÉM QUE IMPLEMENTAR A CLASSE IOperacao

nesta versão, não sendo necessário modificar nada na mesma, ok?

`abstract class Operacoes implements IOperacao` - **abstract**: estamos dizendo que a classe `Operacoes` é uma classe abstrata, ou seja, não queremos criar objetos do tipo `Operacoes`, em nossa **calculadora**, o usuário **sempre estará** fazendo uma **operação**, no entanto, essa **operação** sempre **precisará** ser informada **para que o** objeto correto seja criado, **podendo ele ser** Soma, Subtracao, Multiplicacao ou Divisao, pois cada uma dessas classes saberá como realizar seus cálculos.

implements IOperacao: isso **não** é necessariamente **novo** para você pois já o fizemos na **Atividade** anterior. A **questão nova aqui** é que a classe `Operacoes` é que **passou a implementar** a nossa interface. Quando você **visualizar** o **diagrama de classes**, verá que **entre** a nossa interface (`IOperacao`) e as classes de operações, **teremos** a nossa classe mãe (`Operacoes`).

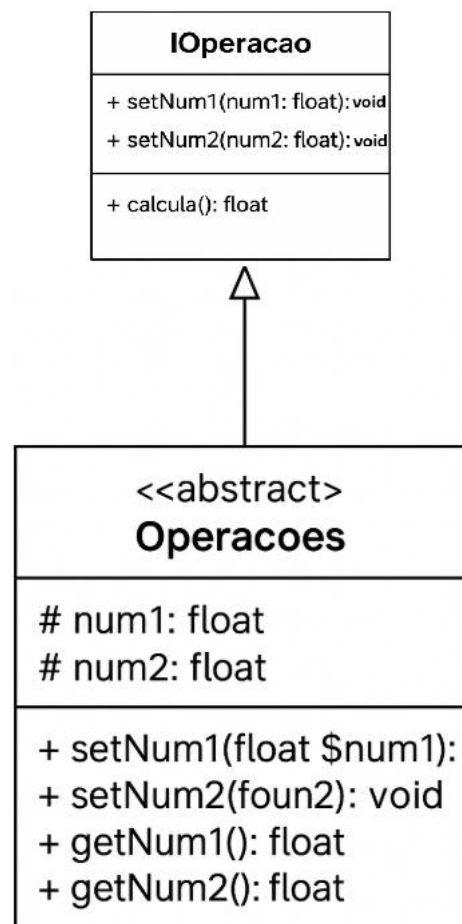
VISUALIANDO AS DUAS CLASSES



Como a classe mãe `Operacoes` é **abstrata** e está **implementando** nossa interface `IOperacao`, **não** é obrigatório que ela **contenha** o método `calcula()` assim como **não** é obrigatório que ela **contenha** nenhuma **implementação** que a interface `IOperacao` **obrigue**. Veja que em **destaque** na classe `Operacoes`, somente os métodos `setters()` foram **implementados**. **Você deve estar se perguntando como isso é possível** e a resposta é simples: quando **definimos** uma

classe **abstrata**, atribuindo a palavra **abstract** em sua definição, o **PHP** entende que essa classe **não é uma classe que constituirá um objeto dela própria**, ela **existe para servir outras classes**. No nosso caso, a classe **Operacoes** **serve** as classes **Soma**, **Subtracao**, **Multiplicacao** e **Divisao** com os **métodos** e **atributos comuns** a elas. Dessa forma, é **considerado** que: **“quem deve implementar o que a interface determina é quem está abaixo da classe mãe**, ou seja, **suas filhas**, as **classes mais especializadas**, e **não a classe mãe em si**. Contudo, o que já estiver na classe mãe, **é dado como já implementado em suas filhas**”.

VISUALIZANDO AS CLASSES RELACIONADAS



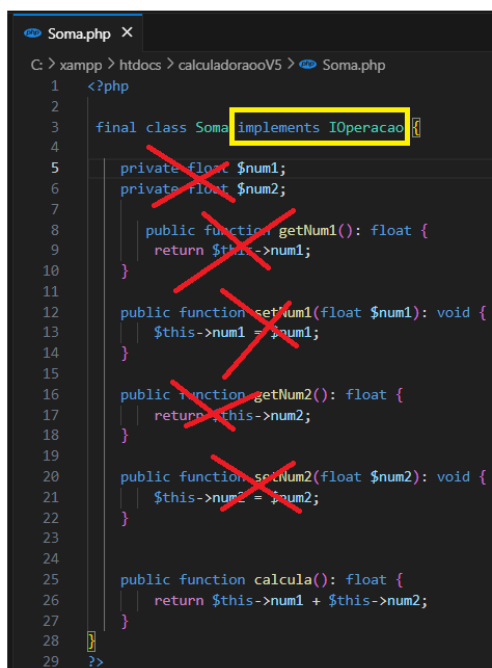
REORGANIZANDO NOSSA HIERARQUIA

SCRIPT CLASSE SOMA

Dando sequência no **desenvolvimento de nossos Scripts**, agora é necessário que **reorganizemos** nossa **hierarquia**, fazendo com que as **classes de operações estendam a classe mãe**, que por sua vez, **implementa a interface**.

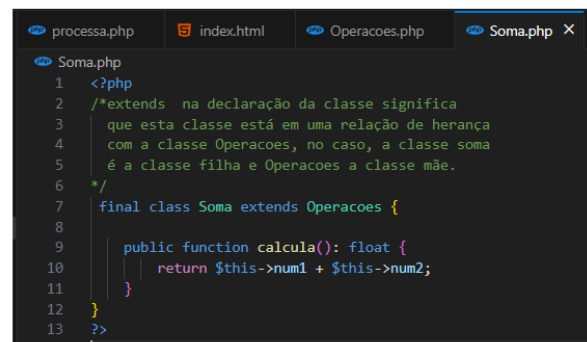
Novamente, tendo como base a classe **Soma**, vamos **escrever o Script desta classe para a versão 6**, atentando para as mudanças necessárias devido a nova arquitetura.

VERSÃO 5



```
1 <?php
2
3 final class Soma implements IOperacao {
4
5     private float $num1;
6     private float $num2;
7
8     public function getNum1(): float {
9         return $this->num1;
10    }
11
12    public function setNum1(float $num1): void {
13        $this->num1 = $num1;
14    }
15
16    public function getNum2(): float {
17        return $this->num2;
18    }
19
20    public function setNum2(float $num2): void {
21        $this->num2 = $num2;
22    }
23
24    public function calcula(): float {
25        return $this->num1 + $this->num2;
26    }
27
28 }
29 ?>
```

VERSÃO 6



```
1 <?php
2 /*extends na declaração da classe significa
3  que esta classe está em uma relação de herança
4  com a classe Operacoes, no caso, a classe soma
5  é a classe filha e Operacoes a classe mãe.
6  */
7 final class Soma extends Operacoes {
8
9     public function calcula(): float {
10         return $this->num1 + $this->num2;
11     }
12 }
13 ?>
```

O QUE MODIFICOU DA VERSÃO 5 PARA A 6 ?

- 1 - Substituir o que está no retângulo amarelo por extends Operações;
- 2 - Remover os atributos \$num1 e \$num2;
- 3 - Remover os métodos getNum1(), setNum1(), getNum2(), setNum2();
- 4 - Manter o método calcula().

IMPORTANTE: a classe **Soma** está **vinculada** com a classe **Operacoes**(classe mãe) por uma relação de **Herança**, ou seja, uma vez que a classe **Operacoes** contém, **a declaração dos atributos que correspondem aos valores que serão calculados, respectivamente \$num1 e \$num2**, e os métodos que armazenam valores nestes atributos, são eles os métodos **setNum1()** e **setNum2()**

respectivamente, esta poderá **compartilhar**, com a **classe Soma**, todos estes recursos fazendo com que filha, no caso Soma, só se preocupe em realizar o cálculo. **Isto deverá ocorrer para as demais classes de operação também.**

SCRIPT CÓDIGO PRINCIPAL

POLIMORFISMO

No início solicitamos que fizesse uma **pesquisa e fichamento** sobre **polimorfismo**. Em suas **pesquisas**, provavelmente, pôde constatar que esse **recurso** trata-se da **capacidade de um objeto assumir várias formas**, ou seja, **a forma de outros objetos**. Obviamente, para que o **polimorfismo** aconteça, é **necessário** que as **classes em questão tenham algum tipo de relacionamento**. **Não conseguimos fazer com que um objeto do tipo String assuma a forma de um objeto do tipo Integer sem que nenhum tipo de conversão seja feita.**

Anteriormente, discutimos um pouco sobre a **classe Operacoes** ser uma classe abstrata e **não constituir por si só um objeto em nosso contexto**. No entanto, todas as **operações aritméticas** que podem ser efetuadas pela calculadora herdam métodos e atributos da classe mãe. Podemos afirmar que a **Soma** é do tipo **Operacoes**, assim como as **demais classes de operações** que temos. Considerando essas afirmações, **poderíamos fazer com que um único objeto assumisse a operação que fosse necessária, de acordo com a escolha do usuário**. Para finalizarmos a **versão 6** de nossa calculadora, vamos fazer a implementação descrita para que possamos discutir melhor todos os conceitos de POO aplicados.

Abaixo a imagem do código principal sendo que, **no geral, não houveram muitas mudanças**, estas, estarão destacadas no retângulo **amarelo** e também cabe salientar que neste momento só estamos contemplando, como deve estar observando, a operação da Soma, **as demais implementações para as demais operações, será por sua conta.**

```

processa.php X index.html Operacoes.php Soma.php IOperacao.php
processa.php
1 <?php
2
3 /*Descarrega a interface Operacoes, o arquivo Soma.php e
4 TrataeMostra.php.
5
6 require_once 'Operacoes.php';
7 require_once 'Soma.php';
8 require_once 'TrataeMostra.php';
9
10 // Recepção dos dados
11
12 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
13
14     //Recebendo os valores e a operação
15
16     $valor1 = $_POST['valor1'] ?? '';
17     $valor2 = $_POST['valor2'] ?? '';
18     $operacao = $_POST['operacao'] ?? '';
19
20     /*Converte e valida os números recebidos a partir da chamada do método estático parse_num()
21     que encontra-se na classe estática Calculadora */
22     $val1 = TrataeMostra::parse_num($valor1);
23     $val2 = TrataeMostra::parse_num($valor2);
24
25     $result = null;
26     $error = null;
27
28     /* Cria o objeto oper e inicializa com null pois
29     este, dependendo da operação escolhida, assumirá
30     como valor esta operação*/
31
32     $oper = null;
33
34     //Verifica se há erro de entrada ou de operação
35
36     if ($val1 === null || $val2 === null) {
37         $error = 'Entrada inválida. Certifique-se de informar números válidos.';
38     } else {
39         /*
40          De acordo com a operação escolhida, criará a instância da classe
41          correspondente a operação escolhida, e a partir daí,
42          por intermédio do objeto da classe, vai acessar os métodos
43          da Classe instanciada para efetuar o cálculo da operação.
44          */
45         switch ($operacao) {
46             case 'soma':
47                 $oper = new Soma();
48                 break;
49
50             //finaliza o switch
51             $oper->setNum1($val1);
52             $oper->setNum2($val2);
53             $result= $oper->calcula();
54             //finaliza o caso
55
56             /* Exibir resultado chamando o método estático exibirResultado()
57             que encontra-se na classe estática Calculadora.
58             */
59             TrataeMostra::exibirResultado($error,$operacao,$val1,$val2,$result);
60         }
61     }
62 }

```

DECOMPONDO !!!!

`require_once 'Operacoes.php';` - importando o conteúdo, neste caso, do arquivo Operacoes.php, que em nosso novo cenário é a nossa classe “mãe”, pois as classes de operações aqui instanciadas estarão vinculadas a esta, portanto, esta importação torna-se extremamente necessário.

`$oper = null;` - declaramos um objeto, que inicialmente recebeu como valor null. Na sequência, dentro do case esse objeto receberá a instância da classe necessária de acordo com o cálculo escolhido pelo usuário, no nosso caso, por enquanto só fizemos para a instância da classe Soma(). Mas, o que isso representa? Observe que, com essa implementação, passamos a trabalhar unicamente com UM objeto em memória, o objeto \$oper, que sempre recebe uma instância diferente, de acordo com cada cálculo escolhido pelo usuário. Na versão anterior, criávamos sempre um novo objeto do tipo Soma, Subtracao, Multiplicacao ou Divisao em memória, sempre que um cálculo era feito. Sendo assim, se antes executássemos 15 cálculos, 15 objetos eram criados em

memória! E agora, se executarmos esses mesmo 15 cálculos, apenas um objeto é criado em memória, o \$oper. Isto por conta de termos utilizado o Polimorfismo de objetos, ou seja, um objeto, no caso \$oper, assumindo várias formas, ou seja, ora instanciando a classe Soma(), ora instanciando a classe Subtracao() e por aí vai, de acordo com a operação escolhida pelo usuário.

```
$oper->setNum1($val1);  
$oper->setNum2($val2);  
$result= $oper->calcula();
```

- Na versão anterior, além do que comentamos acima, também dentro da estrutura switch () case, acabávamos tendo certa repetição de código em relação as chamadas dos métodos setNum1(), setNum2() e calcula(). Agora, temos uma otimização dessas três linhas, **note** que **estão escritas fora da estrutura switch() case** e, sendo assim, **são escritas somente uma vez**, pois **independente da instância** que o objeto **&oper** receber, por intermédio dele estaremos fazendo a operação correta.

ATENÇÃO!!!

Precisamos também dos demais códigos, que são os mesmos das versões anteriores, que você poderá reaproveitar que, por sua vez, servem para limpar/validar os dados recebidos e também os códigos que exibirão o resultado, ambos da classe TrataeMostra, providencie isso.

Execute o programa, TESTE todas as suas funcionalidades e se houver algum erro, procure, primeiramente corrigí-los sozinho, reveja o código repassado e tudo mais, só em último caso recorra ao orientador.

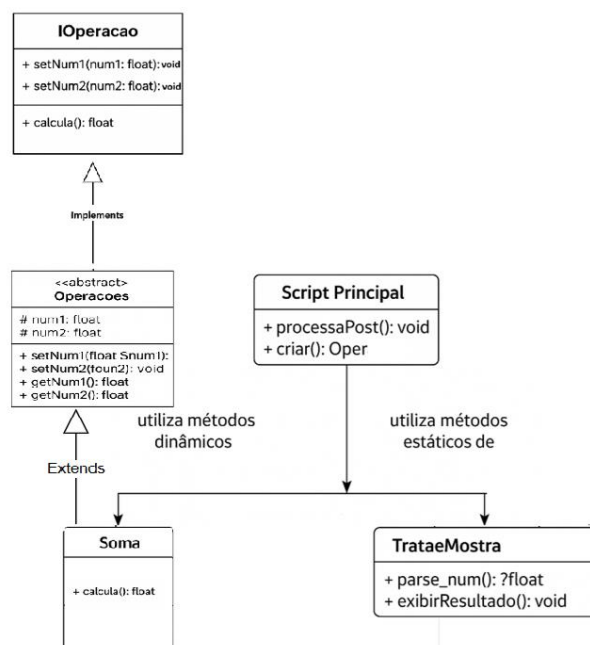
DISCUSSÕES

Nesse momento você já **efetuou** seus testes e já possui **6 versões** da calculadora e também já **conheceu** alguns **conceitos e práticas da Programação Orientada a Objetos**. Nessa versão, como mencionamos no **começo** dessa **Atividade**, nós continuamos mantendo a **divisão de responsabilidade de classes**, agora inserindo uma classe mãe que compartilha atributos e métodos com seus filhos que são as classes das operações, a classe mãe também está vinculada a interface. mantivemos também a classe **TrataeMostra** como estática.

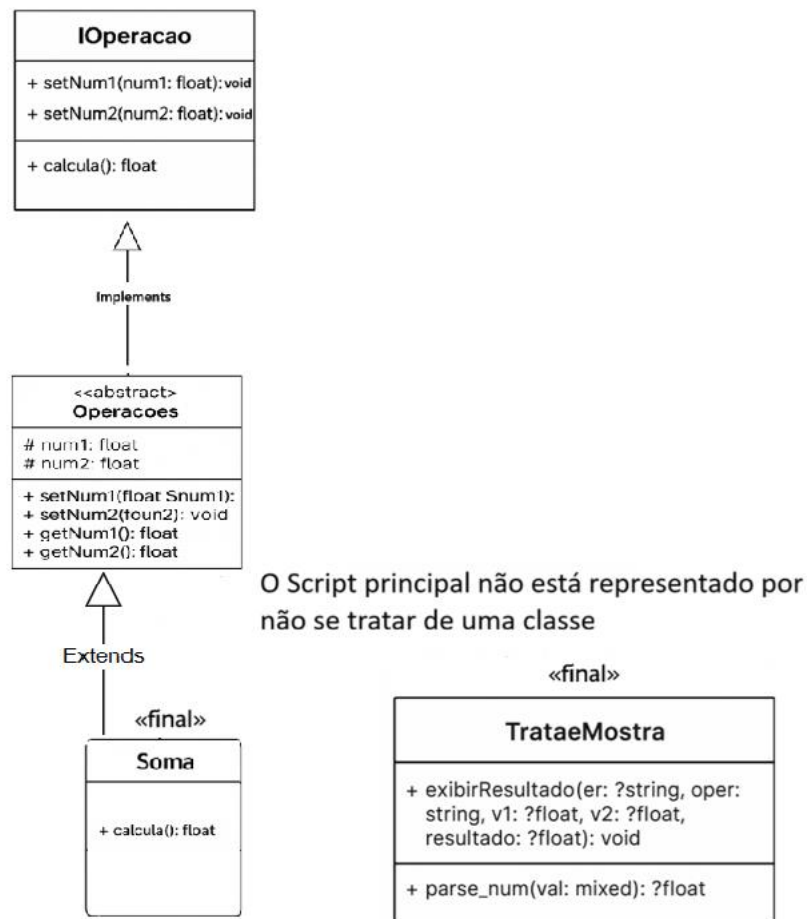
TAREFA

Revise seu fichamento sobre modificadores de acesso e na **validação** discuta com seu orientador o motivo dos atributos **num1** e **num2** terem sido declarados como protegidos na classe mãe **Operacoes**. Além disso, **reflita e tente explicar na validação, com suas palavras, as diferenças entre uma interface e uma classe abstrata**.

REPRESENTAÇÃO GERAL



REPRESENTAÇÃO (UML)



A exemplo do que **fez** anteriormente, construa o **Diagrama de Classe** da nossa solução, baseando-se no Diagrama acima.

Grave o Diagrama na pasta em que salvou o projeto.

CALMA AÍ NOVAMENTE!!!!

LEMBRE-SE QUE ESTA OT APENAS IMPLEMENTOU O VINCULO COM A CLASSE MÃE OPERACAO PARA A OPERAÇÃO SOMA, OK? ADIVINHA QUEM IRÁ IMPLEMENTAR OS DEMAIS VÍNCULOS PARA AS DEMAIS OPERAÇÕES???ISSO MESMO, VOCÊ, ENTÃO, MÃOS A OBRA, E AO FINAL , COMPLEMENTE COM A

**REPRESENTAÇÃO GERAL E O DIAGRAMA UML
ARMAZENANDO-OS NA PASTA DO PROJETO.**

AO TÉRMINO DA IMPLEMENTAÇÃO, execute novamente o programa, **TESTE** todas as suas funcionalidades e se houver algum erro, procure, primeiramente corrigí-los sozinho, reveja o código repassado e tudo mais, só em último caso recorra ao orientador.

**SERÁ QUE EXISTE ALGUMA MANEIRA DE ORGANIZAR
AINDA MAIS NOSSA ESTRUTURA DE ARQUIVO DE NOSSO
PROGRAMA? VEREMOS SE ISSO É POSSÍVEL NA
PRÓXIMA OT.**